

Practical no:4

Name: Gayatri Hemant Pawar

Roll No: 272

Class:SE

Problem Statement: Binary Search Tree -- CO1, CO2, CO3

Implement binary search tree and perform following operations:

- a. Insert (Handle insertion of duplicate entry)
- b. Delete
- c. Search
- d. Display tree (Traversal)
- e. Display - Depth of tree
- f. Display - Mirror image
- g. Create a copy
- h. Display all parent nodes with their child nodes
- i. Display leaf nodes
- j. Display tree level wise

Code:

```
from collections import deque
import copy
```

```
class Node:
```

```
    def __init__(self, x):
```

```
        self.data = x
```

```
        self.left = self.right = None
```

```
    def insert(root, x):
```

```
        if not root:
```

```
            return Node(x)
```

```
        if x < root.data:
```

```
            root.left = insert(root.left, x)
```

```
        elif x > root.data:
```

```
            root.right = insert(root.right, x)
```

else:

print("Duplicate Entry")

return root

def search(root, x):

if not root:

return False

if root.data == x:

return True

return search(root.left, x) if x < root.data else search(root.right, x)

def findMin(root):

while root.left:

root = root.left

return root

def delete(root, x):

if not root:

return root

if x < root.data:

root.left = delete(root.left, x)

elif x > root.data:

root.right = delete(root.right, x)

else:

if not root.left:

return root.right

if not root.right:

return root.left

t = findMin(root.right)

root.data = t.data

root.right = delete(root.right, t.data)

return root

```
def inorder(root):  
    if root:  
        inorder(root.left)  
        print(root.data, end=" ")  
        inorder(root.right)
```

```
def depth(root):  
    if not root:  
        return 0  
    return 1 + max(depth(root.left), depth(root.right))
```

```
def mirror(root):  
    if root:  
        root.left, root.right = root.right, root.left  
        mirror(root.left)  
        mirror(root.right)
```

```
def leaf(root):  
    if root:  
        if not root.left and not root.right:
```

```
            print(root.data, end=" ")  
            leaf(root.left)  
            leaf(root.right)
```

```
def parent(root):  
    if root:  
        print(root.data, "->",  
              root.left.data if root.left else "-",  
              root.right.data if root.right else "-")  
        parent(root.left)  
        parent(root.right)
```

```

def level(root):
    if not root:
        return
    q = deque([root])
    while q:
        t = q.popleft()
        print(t.data, end=" ")
        if t.left:
            q.append(t.left)
        if t.right:
            q.append(t.right)

# Main
root = None

for x in [50, 30, 70, 20, 40, 60, 80]:
    root = insert(root, x)

print("Inorder:")
inorder(root)

print("\nSearch 40:", search(root, 40))

print("Depth:", depth(root))

print("Leaf Nodes:")
leaf(root)

print("\nParent -> Children")
parent(root)

print("Level Order:")
level(root)

```

```
copyTree = copy.deepcopy(root)
print("\nTree Copied Successfully")
```

```
mirror(root)
print("Mirror Inorder:")
inorder(root)
```

```
root = delete(root, 60)
print("\nAfter Deleting 60:")
inorder(root)
```

Output:

Inorder:

20 30 40 50 60 70 80

Search 40: True

Depth: 3

Leaf Nodes:

20 40 60 80

Parent -> Children

50 -> 30 70

30 -> 20 40

20 -> - -

40 -> - -

70 -> 60 80

60 -> - -

80 -> - -

Level Order:

50 30 70 20 40 60 80

Tree Copied Successfully

Mirror Inorder:

80 70 60 50 40 30 20

After Deleting 60:

80 70 50 40 30 20